

Programming a Dense Neural Network

Luka Lambrecht

In this document, a fully functional dense feedforward neural network is programmed from scratch. In the first part, the theory behind the implementation is covered, deriving the formulas for the backpropagation algorithm. This is done starting from an example network with two hidden layers and one output layer. By studying the exact operation of the network on a given input instance, the reverse operation, i.e. finding in which direction the weights of the network should be modified in order to make it better, can be formulated and understood. The second part of this document contains some fragments from a Python implementation, showing how the concepts translate into programmable entities.

Note: only the first part is included for now; the second part remains to be written when I find the time...

1 The math and the concepts

1.1 definitions and naming conventions

An example network is shown below:

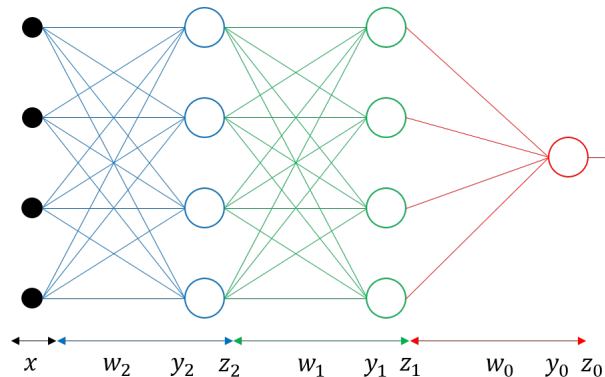


Figure 1: Example network consisting of two hidden layers and one output layer. The number of units in each layer is not representative.

Each layer consists of a linear transformation of the input using its weight matrix and a non-linear transformation of the result. We chose the weight matrix to be of the shape $(ninputs+1, nunits)$, where $ninputs$ is the number of input values and $nunits$ the number of units in that layer. The additional $+1$ in the first dimension allows to take into account a bias term for each unit. These bias terms are not shown in figure 1, but they can be imagined as an additional input line to each unit with a fixed input value of 1. Note that the weight matrix may also be defined as the transpose of the above, in which case the formulas will have to be modified, but the principle remains the same.

1.2 forward pass

The first layer (blue in figure 1) performs a transformation on the input as follows:

$$y_2 = w_2^T \cdot \tilde{x}$$

$$z_2 = f(y_2)$$

where $\tilde{x}$ is the vector x with an additional value of 1 appended at the end to serve as bias term. All x , y and z vectors are considered to be column vectors in the implementation chosen here. This has its implications on the dimensions of the entities in the formula: the dimension of $\tilde{x}$ is $(ninputs+1, 1)$, the dimension of w_2^T is $(nunits, ninputs+1)$, so the dimension of y_2 is $(nunits, 1)$. As f is only an element-wise activation, the dimension of z_2 (the output of this layer) is also $(nunits, 1)$.

It will turn out to be useful to re-write the above formulas in a less compact and more index-based way:

$$[y_2]_i = [w_2^T]_{(i,j)} [\tilde{x}]_j = [w_2]_{(j,i)} [\tilde{x}]_j$$

$$[z_2]_i = f([y_2]_i)$$

The second layer (green in figure 1 performs a similar transformation:

$$y_1 = w_1^T \cdot \tilde{z}_2$$

$$z_1 = f(y_1)$$

or in index notation:

$$\begin{aligned} [y_1]_i &= [w_1^T]_{(i,j)} [\tilde{z}_2]_j = [w_1]_{(j,i)} [\tilde{z}_2]_j \\ [z_1]_i &= f([y_1]_i) \end{aligned}$$

where of course the numbers *ninputs* and *nunits* may be different from the previous layer (as long as *nunits* from the first blue layer equals *ninputs* from the green layer!).

Finally, we have:

$$y_0 = w_0^T \cdot \tilde{z}_1$$

$$z_0 = f(y_0)$$

or in index notation:

$$\begin{aligned} [y_0]_i &= [w_0^T]_{(i,j)} [\tilde{z}_1]_j = [w_0]_{(j,i)} [\tilde{z}_1]_j \\ [z_0]_i &= f([y_0]_i) \end{aligned}$$

where w_0 has dimension $(nunits+1,1)$, i.e. the final layer has only one unit, so that y_0 and z_0 (the network output) are simple scalars.

Notice that during the forward pass, all intermediate y - and z -vectors must be stored as they will be needed in the backward pass.

1.3 backward pass

Now the gradient of the error with respect to all the weight matrices of the network can be calculated. This is in essence nothing more than an application of the chain rule from calculus. We start with w_0 :

$$\frac{\partial E}{\partial [w_0]_i} = \frac{\partial E}{\partial z_0} \frac{\partial z_0}{\partial y_0} \frac{\partial y_0}{\partial [w_0]_i}$$

The first factor on the right hand side depends on the error measure. For example, if we use the mean-square-error, we have (for a single instance):

$$E = \frac{1}{2}(t - z_0)^2$$

where t is the label and z_0 the prediction. In that case:

$$\frac{\partial E}{\partial z_0} = z_0 - t$$

Other error measures will give rise to other expressions, but the important point is that it is a scalar depending only on the network output and the label of the instance under consideration. The second factor is simply the derivative of the activation function f evaluated in the inactivated scalar y_0 . Again the exact expression depends on the choice of f , but the result is a scalar depending only on y_0 . The third and final factor is calculated as follows:

$$y_0 = [w_0]_j [\tilde{z}_1]_j \quad \rightarrow \quad \frac{\partial y_0}{\partial [w_0]_i} = [\tilde{z}_1]_j$$

The final result can be re-translated into matrix notation:

$$\frac{\partial E}{\partial w_0} = \tilde{z}_1 f'(y_0) \frac{\partial E}{\partial z_0}$$

where the last two factors are scalars and both the left hand side and the first factor on the right hand side are column vectors of size $nunits+1$ (of the green layer) = $ninputs+1$ (of the red layer).

The same procedure can be applied to w_1 :

$$\frac{\partial E}{\partial [w_1]_{(i,j)}} = \frac{\partial E}{\partial z_0} \frac{\partial z_0}{\partial y_0} \frac{\partial y_0}{\partial [z_1]_k} \frac{\partial [z_1]_k}{\partial [y_1]_l} \frac{\partial [y_1]_l}{\partial [w_1]_{i,j}}$$

Using the formulas from the forward pass, this translates into:

$$\begin{aligned} \frac{\partial E}{\partial [w_1]_{(i,j)}} &= \frac{\partial E}{\partial z_0} f'(y_0) [w_0]_k f'([y_1]_l) \delta_{kl} [\tilde{z}_2]_i \delta_{lj} \\ &= \frac{\partial E}{\partial z_0} f'(y_0) [w_0]_j f'([y_1]_j) [\tilde{z}_2]_i \end{aligned}$$

Again, this can be re-written in matrix notation:

$$\frac{\partial E}{\partial w_1} = \tilde{z}_2 f'(y_0) \frac{\partial E}{\partial z_0} w_0^T \text{diag}(f'(y_1))$$

A note on dimensions: $\tilde{z}_2$ is an $(ninputs+1,1)$ matrix, $f'(y_0) \frac{\partial E}{\partial z_0}$ is scalar and w_0^T is an $(1,nunits)$ matrix. In fact w_0 is a $(nunits+1,1)$ matrix, but a careful inspection of the indices shows that the last index, corresponding to the weight of the bias term, should not appear in the formula, so we cut it out from w_0 in this case. Also note that in the above lines, $ninputs$ and $nunits$ refer to the green layer in figure 1!

Finally, the same procedure can be applied to w_2 . From this result, we will be able to generalize the result to an arbitrary number of layers. We have:

$$\frac{\partial E}{\partial [w_2]_{(i,j)}} = \frac{\partial E}{\partial z_0} \frac{\partial z_0}{\partial y_0} \frac{\partial y_0}{\partial [z_1]_k} \frac{\partial [z_1]_k}{\partial [y_1]_l} \frac{\partial [z_2]_m}{\partial [y_2]_n} \frac{\partial [y_2]_n}{\partial [w_2]_{(i,j)}}$$

We work out all the parts separately:

- factor 1 and 2: $\frac{\partial E}{\partial z_0} \frac{\partial z_0}{\partial y_0} = \frac{\partial E}{\partial z_0} f'(y_0)$ (see above).
- factor 3: $\frac{\partial y_0}{\partial [z_1]_k} = [w_0]_k$
- factor 4: $\frac{\partial [z_1]_k}{\partial [y_1]_l} = f'([y_1]_l) \delta_{kl}$
- factor 5: $\frac{\partial [y_1]_l}{\partial [z_2]_m} = [w_1]_{(m,l)}$
- factor 6: $\frac{\partial [z_2]_m}{\partial [y_2]_n} = f'([y_2]_n) \delta_{ml}$
- factor 7: $\frac{\partial [y_2]_n}{\partial [w_2]_{(i,j)}} = [\tilde{x}]_i \delta_{nj}$

All these results follow directly from the index expressions from the forward pass. Putting things back together, we find:

$$\frac{\partial E}{\partial [w_2]_{(i,j)}} = \frac{\partial E}{\partial z_0} f'(y_0) [w_0]_k f'([y_1]_k) [w_1]_{(j,k)} f'([y_2]_j) [\tilde{x}]_i$$

Or in matrix notation:

$$\frac{\partial E}{\partial w_2} = \tilde{x} \frac{\partial E}{\partial z_0} f'(y_0) w_0^T \text{diag}(f'(y_1)) w_1^T \text{diag}(f'(y_2))$$

Notice that again, the last row of w_0 and w_1 must be pruned before using them in the formula.

1.4 the backpropagation algorithm

The last result from the previous section is easily generalized to an arbitrary number of layers. Now we are ready to calculate the gradient of the error with respect to all weights in the network, given a single instance and a set of weight matrices (initialized randomly). The first step consists of straightforwardly applying the formulas from the forward pass and storing all intermediate y - and z -vectors. For the backward pass, we start from a (scalar) seed $\frac{\partial E}{\partial z_0} f'(y_0)$. The gradient with respect to w_0 is obtained by (left-)multiplying this seed with a termination z_1 . Next, the seed is modified by (right-)multiplying it with $w_0^T \text{diag}(f'(y_1))$ (a row-vector). The gradient with respect to w_1 is obtained by left-multiplying the seed with a termination z_2 . The multiplication is a matrix multiplication of a column-vector with a row-vector, thus resulting in a matrix (the correctness of the dimensions can be checked). Next, the seed is modified by right-multiplying it with $w_1^T \text{diag}(f'(y_2))$ (a matrix), the result of which is again a row-vector. This procedure can be arbitrarily extended. The bias terms are only taken into account in the termination (using e.g. $\tilde{z}_1$ instead of z_1); the weight matrices in the modification of the seed must be pruned to leave out the bias weights. These rules of the thumb follow from the index expressions in the previous paragraph.

These calculations can be repeated for a batch of training instances, adding or averaging the gradients obtained for each instance separately. Next, the weights are modified using a certain update rule, e.g. gradient descent:

$$w_i \leftarrow w_i - \alpha \frac{\partial E}{\partial w_i}$$